Audify

AI-Powered Environmental Sound Classification

Deep Learning · Audio Signal Processing · GPU Deployment

Python | PyTorch | Torchaudio | Librosa

Modal (GPU) | FastAPI | TensorBoard | ResNet CNN

01 — Project Overview

Audify is an end-to-end deep learning system that classifies real-world environmental sounds into 50 categories using a custom ResNet-style Convolutional Neural Network. The pipeline covers everything from raw audio ingestion and feature extraction to model training on cloud GPUs, production inference via a REST API, and rich visualisation of internal network activations — making the model's decision-making fully transparent.

50	2,000	5 sec	A10G
Sound Classes	Audio Clips	Clip Duration	GPU (Modal)

02 — Application Interface

The web interface allows users to upload a WAV audio file and instantly receive ranked predictions from the deployed model. The clean drag-and-drop design keeps interaction frictionless while the GPU-backed API handles inference in the background.

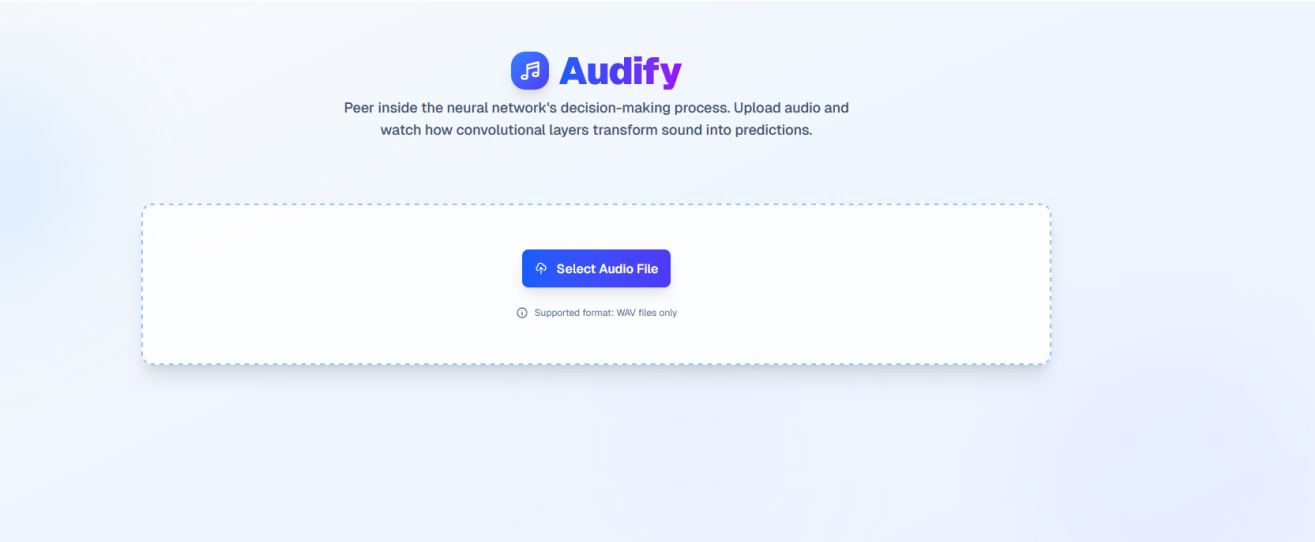

Figure 1 — Audify upload interface: users drop a WAV file to trigger real-time inference

03 — Classification Results

After inference the model returns the Top-5 predicted sound classes alongside confidence scores and a colour-coded progress bar. In the example below, **Chirping Birds** is identified with 96.9% confidence — demonstrating the model's strong discriminative capability on the ESC-50 benchmark.

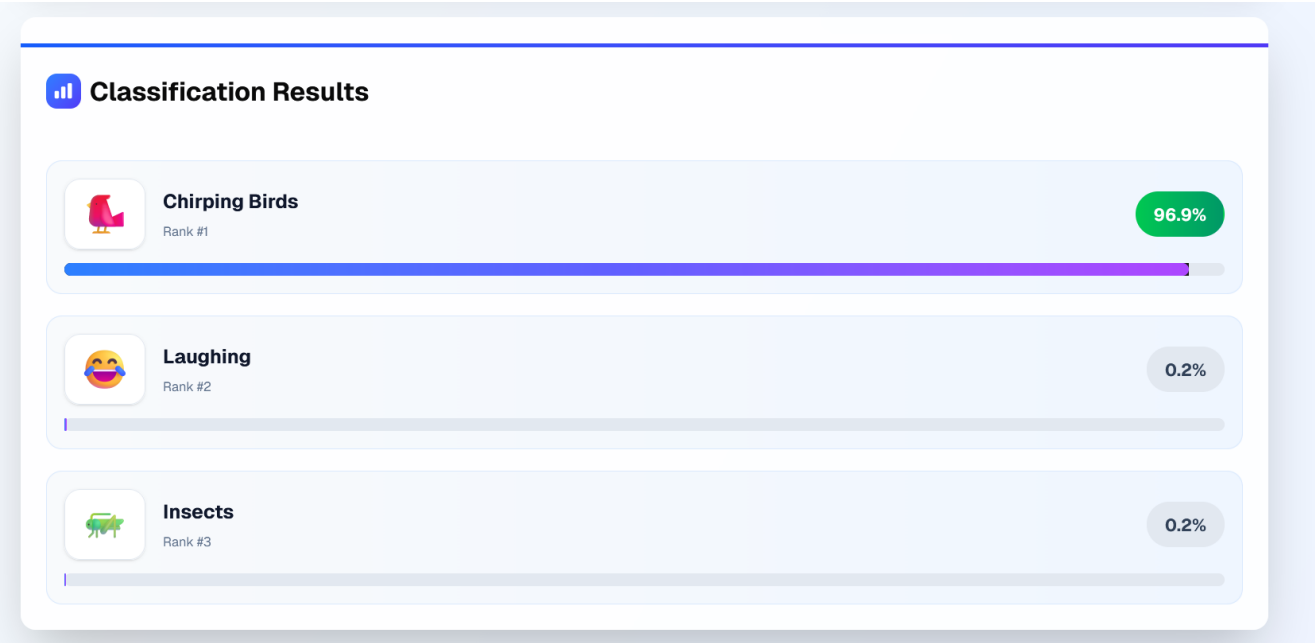

Figure 2 — Top-5 classification output with ranked confidence scores

04 — Audio Representation

Raw audio is first converted to a **Log Mel-Spectrogram** (128 mel bands, n_fft=2048, hop_length=512, sample rate 44 100 Hz) before entering the network. The UI simultaneously renders both the spectrogram heat-map and the raw waveform, giving immediate visual feedback on the input signal quality.

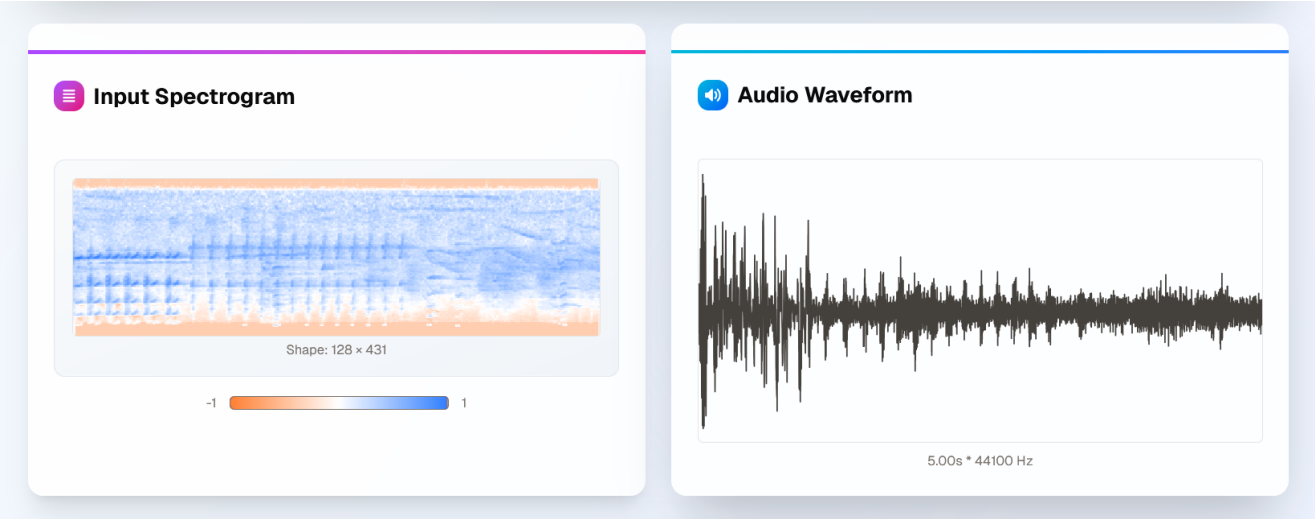

Figure 3 — Input Mel-Spectrogram (128×431) and raw audio waveform (5 s @ 44 100 Hz)

05 — Model Architecture

The backbone is a custom **ResNet-style CNN** built entirely in PyTorch. Four residual layer groups (3 / 4 / 6 / 3 blocks) progressively downsample the spectrogram from 32×108 to 4×14 while expanding channel depth from 64 → 512. Batch normalisation and skip connections in every block ensure stable gradient flow during the 150-epoch training run.

Conv1 + MaxPool	—	1 → 64	32 × 108
Layer 1	3 blocks	64	32 × 108
Layer 2	4 blocks	64 → 128	16 × 54
Layer 3	6 blocks	128 → 256	8 × 27
Layer 4	3 blocks	256 → 512	4 × 14
AvgPool + FC	—	512 → 50	1 × 1

06 — CNN Feature Map Visualisation

A key differentiator of Audify is its **model interpretability** module. Every intermediate feature map — from conv1 through to layer4 — is extracted during inference and returned to the UI for visualisation. The heat-maps reveal how successive residual layers progressively abstract low-level frequency patterns into high-level audio semantics, making the network's reasoning fully transparent.

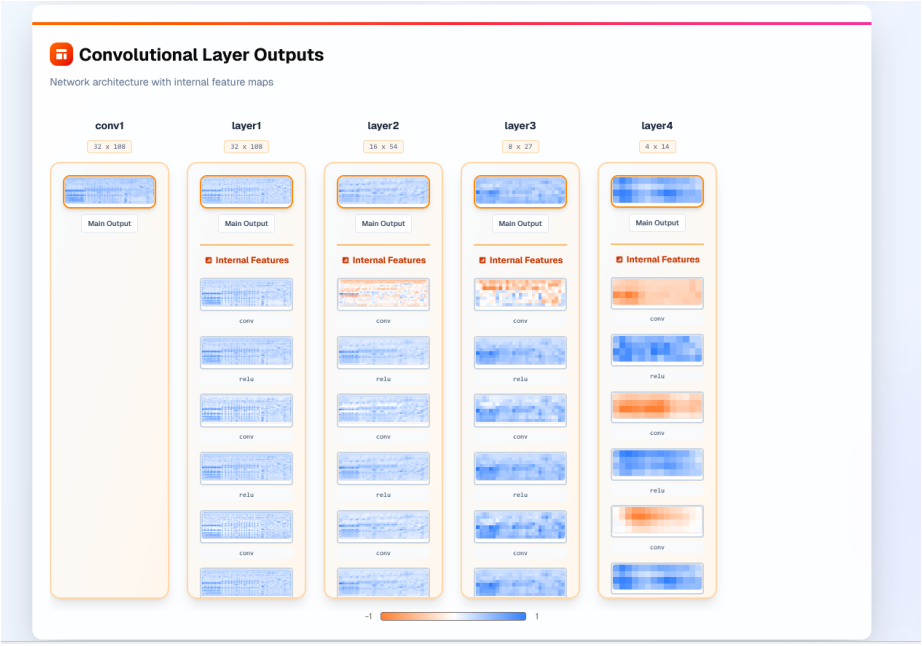

Figure 4 — Convolutional layer outputs across all 5 stages (conv1 → layer4); spatial resolution decreases from 32×108 to 4×14 as abstraction increases

07 — Training & Monitoring

Training was conducted on a **Modal A10G GPU** for up to 150 epochs with early stopping (patience = 20). The optimizer was **AdamW** (lr = 3e-4, weight decay = 0.01) paired with a **OneCycleLR** scheduler and gradient clipping (max norm = 1.0). Data augmentation included **MixUp** (Beta 0.4) and **SpecAugment** (time + frequency masking). Cross-entropy loss with label smoothing (0.1) was used throughout. All metrics were

logged live to **TensorBoard**.

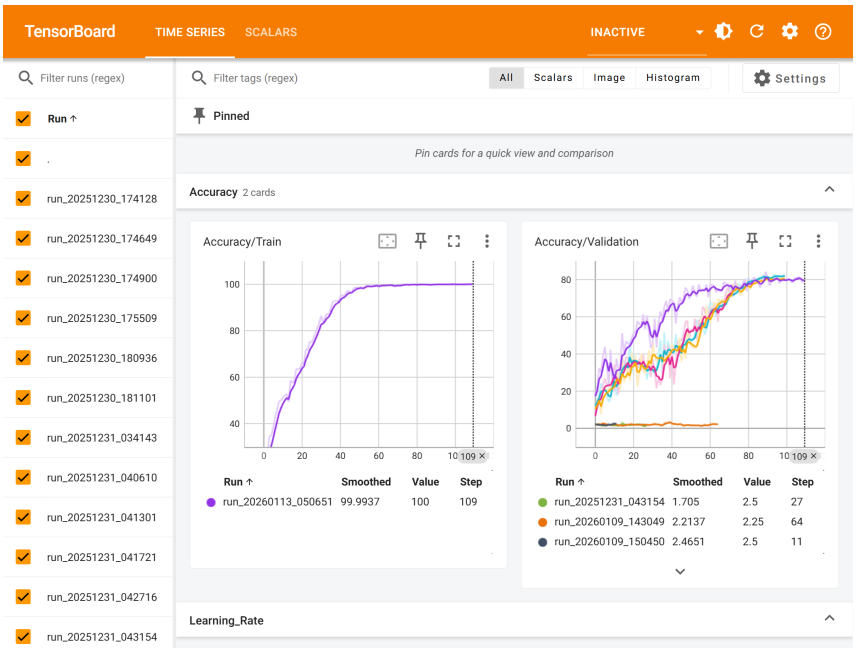

Figure 5 — TensorBoard dashboard: training accuracy converges to ~100% while validation accuracy reaches ~80%+ across multiple runs

08 — Tech Stack

Category	Tools / Libraries
Deep Learning	PyTorch, Custom ResNet CNN, MixUp, SpecAugment
Audio Processing	Torchaudio, Librosa, SoundFile (44 100 Hz, Mel Spectrograms)
Training Infra	Modal (A10G GPU), TensorBoard, OneCycleLR, AdamW
Inference API	FastAPI, Pydantic, Modal FastAPI endpoint
Deployment	Modal cloud GPU, scalable REST API, base64 audio encoding
Language	Python 3